

OSINT-MERCADO DAEMON

PRODUCT REQUIREMENTS DOCUMENT (PRD) & ARCHITECTURE DECISION RECORDS (ADRS)

1. EXECUTIVE SUMMARY

The OSINT-Mercado Daemon is a local-first desktop application designed to systematically audit and expose structural overpricing in municipal procurement across Chile. By ingesting public purchase orders from the Mercado Público / ChileCompra APIs and cross-referencing them against scraped local retail baselines, the software identifies statistical pricing anomalies. Designed for analytical independence, it requires zero community input or volunteer sourcing, functioning as a standalone watchdog utility.

2. PROBLEM STATEMENT & SCOPE

The Gap: While state procurement data is transparent and available via API, it is highly unparsed and dense. Minor corruption or extreme administrative waste at the local municipal level (e.g., purchasing standard electronics or office supplies at 300% markup) goes largely unnoticed without dedicated manual auditing.

Primary Objective: Automate anomaly detection for localized municipal acquisitions of standard, commoditized goods.

Out of Scope: Auditing complex service contracts, public works (Obras Públicas), or specialized consulting where pricing lacks a standardized retail baseline.

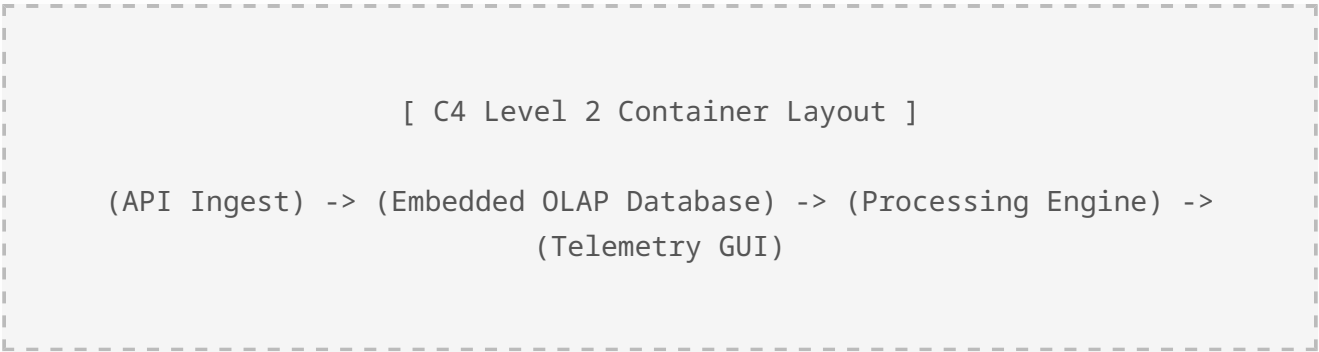
2.1 Core Functional Requirements

- **State Data Ingestion:** Automated, scheduled fetching from the Mercado Público API, filtered strictly by municipal buyer codes.
- **Retail Baseline Engine:** A background scraper that periodically indexes standard prices for a predefined basket of goods from major Chilean hardware, tech, and wholesale distributors.
- **Anomaly Detection Core:** A statistical cross-referencing engine that flags purchase orders exceeding a configurable deviation threshold compared to the retail median.
- **Dashboard Telemetry:** A real-time data visualization interface for filtering anomalies by severity, commune, and product category.

3. SYSTEM ARCHITECTURE

The system is designed with a focus on local execution, high performance, and visual clarity, mapped conceptually through a C4 model approach.

- **Level 1 (Context):** The Daemon mediates between the local User (Auditor) and public external systems (Mercado Público APIs, Retail targets).
- **Level 2 (Container):** Comprises an API Poller, a high-speed Analytical Engine, an embedded Data Store, and an Immediate-Mode UI for telemetry.



4. ARCHITECTURE DECISION RECORDS (ADRS)

ADR 001: Core Data Processing Engine

Status: **Accepted** | Context: High-velocity local parsing and dataframe joins.

Decision: The data pipeline will be built in `Python` utilizing `Polars` for DataFrame manipulation, with custom heavy-lifting algorithms implemented in `Rust` (via PyO3).

Rationale: Cross-referencing thousands of unstandardized municipal purchase strings against scraped retail strings requires intensive fuzzy matching. Standard local processing frameworks struggle with this volume in real-time. By utilizing Polars for multi-threaded data structuring and dropping down to Rust via PyO3 for the specific fuzzy-matching logic, the daemon can parse daily national municipal data in milliseconds, keeping the desktop footprint minimal while maximizing performance.

ADR 002: Desktop Dashboard GUI

Status: **Accepted** | Context: Rendering complex, real-time data anomalies without UI lockup.

Decision: The user interface will be implemented using `Dear PyGui`.

Rationale: Traditional Python desktop frameworks often struggle when tasked with rendering highly dynamic telemetry grids or scatter plots with thousands of outlier points. Dear PyGui provides an immediate-mode, GPU-accelerated rendering pipeline. This ensures that the anomaly dashboard remains completely fluid and responsive, even when rendering dense statistical visualizations of overpricing trends.

ADR 003: Embedded Analytics Database

Status: **Accepted** | Context: Storing historical baselines without external server dependencies.

Decision: `DuckDB` will be used as the local analytical backend.

Rationale: Maintaining the "zero-dependency" requirement means no background database servers can be expected on the user's machine. DuckDB operates entirely in-process, integrates natively with Polars, and is highly optimized for OLAP (analytical) queries, making it ideal for querying historical price baseline data locally.

5. FUTURE TECHNICAL ROADMAP

Once the core anomaly detection pipeline is stable, future iterations will focus on exporting the flagged datasets into standardized, verifiable reports that can be directly submitted to local oversight bodies (Contraloría) with cryptographic hashing to prove data provenance from the official state APIs.